

Architecture, Stack, and Roadmap

The complete technical plan for the myStockMarket command center: system architecture, a justified technology stack, data sources with independently verified pricing, the after-close pipeline, data models, a phased roadmap, and risks. Companion to the Research Report, whose evidence constraints this design implements.

“Build the whole thing on the stack you already know, with one deliberate departure: the nightly pipeline runs as a scheduled GitHub Actions workflow, not on Vercel. Expected all-in cost: a couple of coffees a month — all of it LLM tokens.”

— The one-paragraph conclusion of this blueprint

Contents

01	Decisions at a glance	the whole stack on one page, plus the assumptions it rests on
02	System architecture	the seven boxes, the nightly flow, and the timezone math
03	Stack decisions in detail	every layer: choice, why, alternatives, tradeoffs
04	The data plan	a verified \$0 baseline and a pre-researched paid upgrade ladder
05	The AI layer	extract → synthesize → verify, with cost math and hallucination gates
06	Data model	serving tables in Postgres, history in Parquet
07	Security, operations & cost	auth, secrets, monitoring, and the monthly bill
08	Implementation roadmap	seven phases with acceptance criteria
09	Risks & mitigations	what breaks, how you find out, what you do
10	Sources	official pages used for verification, with dates

Decisions at a glance

One deliberate departure from the familiar path, one architectural split, and a bill that rounds to a coffee. Everything else is the stack you already ship with.

1.1 The shape of the system

The app is **read-mostly by construction**: a Python pipeline does all the work in the cloud each evening after the close — ingest, scan, compute base rates, write the briefing — and the web app just renders the results. That single decision collapses most architectural questions: no always-on backend, no queues, no websockets on day one, no server for you to babysit. The one departure from your Vercel-native habits is forced by physics: Vercel Hobby caps every function at 300 seconds and fights pandas-sized Python bundles, while a scheduled **GitHub Actions** workflow in a private repo gives a full Python environment, 6-hour job limits, and free minutes — and the wide after-close window (the pipeline needs barely an hour of it before your evening read) absorbs its cron jitter with room to spare.

STACK AT A GLANCE

Layer	Choice	One-line why	Cost
Frontend	Next.js (App Router) + TypeScript + Tailwind + shadcn/ui	Your core skill set; server components read serving tables directly.	\$0
Backend	None — thin route handlers only	The pipeline pre-computes everything; a separate API is pure ops overhead for one user.	\$0
Serving DB	Supabase free Postgres (500MB) via Prisma	You know it; nightly pipeline writes double as the anti-pause heartbeat. Neon is the drop-in alternative.	\$0
History store	Parquet on Cloudflare R2, queried by DuckDB/Polars	~12.6M rows ≈ 150–400MB scans in seconds; Postgres adds ceremony for zero benefit. R2 is a cache, not a system of record.	\$0
Pipeline	Python 3.12 · Polars + DuckDB + exchange_calendars + httpx	The right tool for columnar scans; indicators hand-rolled as ~200 lines of tested Polars expressions.	\$0
Scheduler	GitHub Actions cron 37 22 * * 1-5 + manual dispatch	Full Python env, 6h limit, free private-repo minutes; jitter is irrelevant in a multi-hour window.	\$0
Charts	TradingView Lightweight Charts v5 (Apache-2.0)	Purpose-built candlestick engine; panes, markers, price lines for explain-why annotations. Keep the attribution logo.	\$0
General viz	Recharts 3 (shadcn charts); tree-shaken ECharts 6 later if a heatmap is needed	Idiomatic JSX for sparklines/small multiples; defer the heatmap engine until the need is real.	\$0
AI	Anthropic API — Haiku-class extract → Sonnet-class synthesize → deterministic verify; Batch API by default	The pipeline computes all numbers; the LLM narrates them verbatim under a deterministic publish gate.	~\$7–11/mo

Layer	Choice	One-line why	Cost
Hosting	Vercel Hobby	Thin read layer; non-commercial personal use is exactly what Hobby is for.	\$0
Auth	HTTP Basic Auth in middleware (proxy) + CRON_SECRET bearer on pipeline routes	One user over HTTPS; also the licensing compliance mechanism (data stays private to you).	\$0
Monitoring	Actions failure emails + healthchecks.io dead-man ping	The catastrophic failure is a silently dead cron; absence-of-success must itself alert.	\$0

Expected monthly cost \$7–11 entirely LLM tokens; ~\$7 typical	Nightly LLM spend \$0.33 extraction batched; \$0.51 if the all-sync fallback fires	Briefing ready by ~8:40pm EDT typical (7:40pm EST) · promise 9:00pm ET year-round · >12h before the next open	Paid services required 0 every non-LLM tier verified free, Jul 2026
--	--	--	---

1.2 Assumptions this blueprint rests on

Assumption	Consequence if it changes
US equities — NYSE & Nasdaq — are the market	US coverage drives every provider choice, and all timing anchors to the US session: pre-market 4:00–9:30am ET, regular 9:30am–4:00pm ET, after-hours 4:00–8:00pm ET; catalysts are the US earnings calendar and US macro prints (FOMC, CPI, jobs report). Any other market would be a separate ingest adapter behind the same repository interface — none is planned.
Single user (you), single household, non-commercial	Every data license and the Vercel Hobby terms depend on this. Showing the dashboard to others breaches provider ToS — the Basic Auth wall is the compliance mechanism.
You live on market time — Long Island, NY (America/New_York, DST observed); nothing runs on your hardware	The pipeline runs in the cloud on UTC-fixed crons — deliberately DST-proof: your wall clock shifts twice a year, the triggers never do. 22:37 UTC lands 6:37pm EDT / 5:37pm EST — comfortably after the 4:00pm close in both regimes, so the schedule needs no seasonal edit, ever.
Watch/paper-first; no brokerage integration in v1	A pedagogical stance (Research Report §6.4) — build a graded paper track record before real dollars — that also removes an entire class of security risk from v1. As a US resident you have full brokerage access, so live integration is a legitimate later phase, gated on the paper record, not on legal review.
Free-first data; pay only when specific friction appears	The upgrade ladder in Part 4 is pre-researched so a paid migration is a same-week move, not a scramble.
Daily bars are sufficient for v1 (no intraday history)	Minute bars multiply storage ~400x and change the Parquet layout; the design isolates that decision behind the ingest adapter.
Budget tolerance ≈ \$15/mo now, ~\$50–75/mo if upgrades fire	Cost table in Part 7; each upgrade is adopted singly, triggered by named friction.

NON-NEGOTIABLES INHERITED FROM THE RESEARCH REPORT

No directional forecasts anywhere in the system. Every base rate ships with sample size and a Wilson interval, computed in the pipeline — never by the LLM. The LLM narrates pre-computed numbers verbatim behind a deterministic verification gate. Signals are logged append-only from the first phase so the track record is real. No trending/most-bought surfaces. Paper-first.

System architecture

Seven boxes, one direction of flow. Everything heavy happens in two short cloud jobs in your early evening, right after the close; the app is a thin, authenticated window onto their output. Every trigger is fixed in UTC — DST-proof by construction: when your clocks shift (you live on US Eastern), the triggers don't move, and they sit after the close in both EDT and EST.

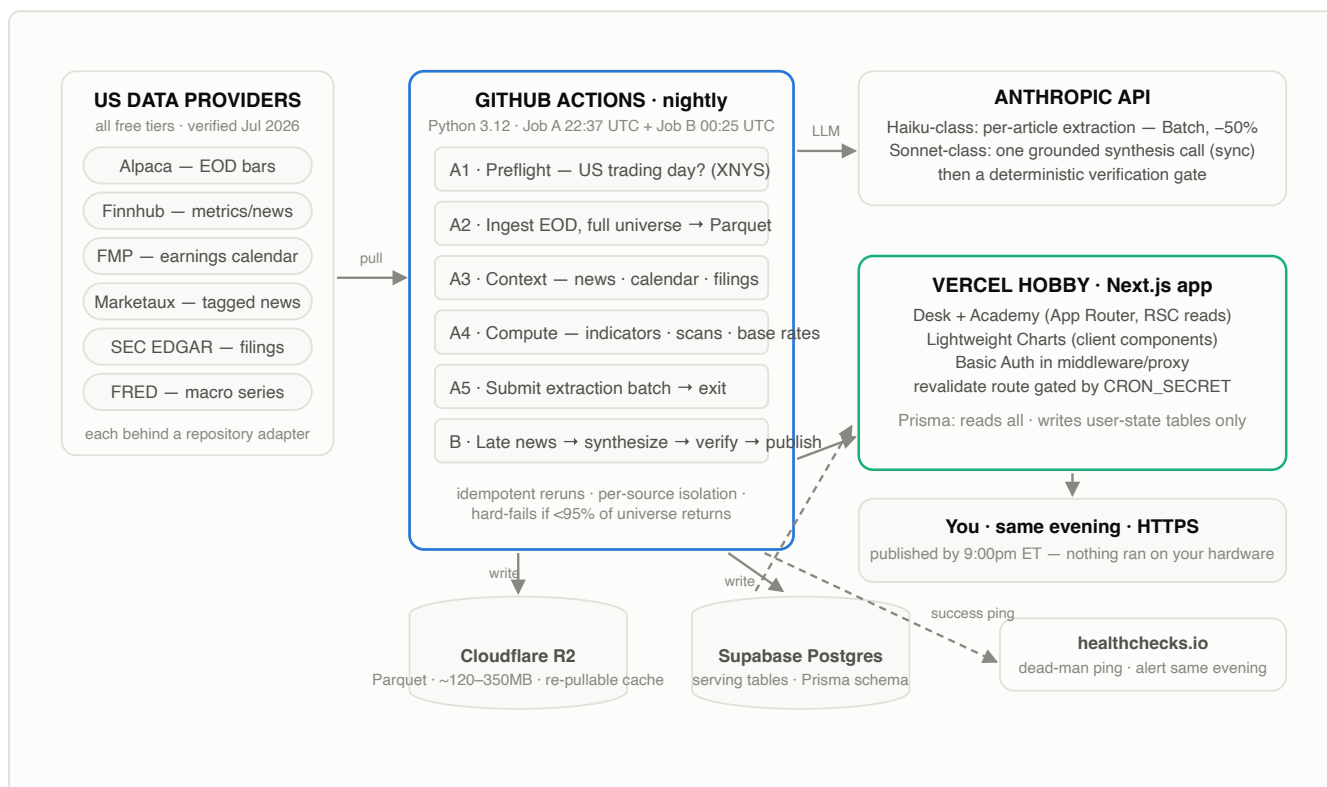


Figure 2.1 — System architecture. Solid arrows are the nightly batch flow; dashed arrows are the publish/monitoring signals. Two short GitHub Actions jobs do all heavy work; the app reads everything and writes only user-state tables (watchlist, journal, review state, paper trades).

2.1 An evening in the life of the pipeline

All triggers are **fixed in UTC**; your clock is US Eastern, which observes DST. So the local times below shift by an hour between summer and winter while the UTC triggers stay put — and both regimes clear the close comfortably. The design splits the evening into two short jobs so no runner ever sits idle waiting on the LLM batch.

EVENING SCHEDULE — UTC-ANCHORED, DST-PROOF. YOUR-CLOCK COLUMN SHOWS EDT (SUMMER); ON THE WINTER CLOCK (EST) EVERY LOCAL TIME BELOW READS ONE HOUR EARLIER.

Stage	What happens	UTC	Your clock (EDT)
-------	--------------	-----	------------------

Stage	What happens	UTC	Your clock (EDT)
US regular session (context)	NYSE/Nasdaq regular hours 9:30am–4:00pm ET — your own working day; pre-market from 4:00am ET, after-hours until 8:00pm ET. Most earnings land before the open (bmo) or just after the close (amc, ~4:00–5:30pm ET).	13:30–20:00 (EDT) 14:30–21:00 (EST)	9:30am–4:00pm (ET-defined, both regimes)
Job A · trigger + preflight	Cron <code>37 22 * * 1-5</code> fires mid-after-hours (6:37pm EDT / 5:37pm EST): regular-session bars are final, amc earnings are out. <code>exchange_calendars</code> (XNYS) exits cleanly — with a success ping — on US holidays. The :37 minute dodges GitHub's top-of-hour cron congestion; worst-case ~30-min jitter is absorbed by the schedule's slack.	22:37	6:37pm
A2 · Full-universe EOD ingest	The day's daily bars for the defined universe (~5–6k NYSE/Nasdaq/AMEX symbols) from Alpaca's multi-symbol bars endpoints, well inside 200 calls/min; row-count validation; current-year Parquet partition rewritten in R2. Weekly: corporate-actions refresh — any split or large dividend rewrites the affected symbol's rows <i>across all year partitions</i> , plus a trailing-5-session re-pull for late corrections.	22:40–22:55	6:40–6:55pm
A3 · Context ingest, sweep #1	Rate-limit-aware parallel pulls: Finnhub metrics + company news (60/min), FMP US earnings calendar with bmo/amc flags (250/day budget), SEC EDGAR per-CIK submissions (≤8 req/s, declared User-Agent), FRED series + release calendar (2 req/s), Marketaux tagged news. Each source isolated — one failure degrades one briefing section, never the run.	22:40–23:05	6:40–7:05pm
A4 · Compute	DuckDB attaches the Parquet history; Polars computes ~15–20 vectorized indicators; SQL window queries produce screens, base rates (signals shifted one bar to kill lookahead), volatility bands, and the computed-stats table the LLM will narrate verbatim.	23:05–23:15	7:05–7:15pm
A5 · Submit + exit	Extraction batch (Stage A of Part 5) submitted to the Batch API; scan/serving staging written; Job A exits. Runner minutes stop here — nobody waits on the batch.	~23:20	~7:20pm
Job B · finish + publish	Second cron (<code>25 0 * * 2-6</code>) wakes ~65 min after submission: a late-news delta sweep (after-hours is fully closed in summer, and US earnings calls are done even in winter), batch results fetched (typically complete long before), any delta items extracted synchronously, then <i>one</i> synchronous synthesis call, the deterministic verification gate, the single-transaction publish, revalidate, and the dead-man success ping.	00:25–00:40	8:25–8:40pm
Fallback path	If the extraction batch is still incomplete at 00:40 UTC (Job B gives collection 15 minutes): cancel it (already-processed items still bill) and extract the remainder synchronously (~3 min at list price).	worst ~00:50	worst ~8:50pm
Your evening	Briefing on screen — the completed session already graded, tomorrow prepared — by 9:00pm ET year-round, read tonight or over pre-open coffee. Guaranteed because every trigger above is UTC-fixed.	≤00:50	by 9:00pm

WHY THE TIMEZONE MATH IS FRIENDLY

You live on market time, so there is no conversion to reason about — the close is 4:00pm on your own clock, Job A fires 1h37m–2h37m later (bars settled in both DST regimes), and the verified briefing publishes before 9:00pm the same evening: after-hours noise largely done, more than twelve hours before the next open. Keeping the crons UTC-fixed is still the right call — DST transitions can never skip or double a run; the only thing that moves is the label on your wall clock (an hour earlier in winter). The “briefing” is honestly an evening digest plus next-session preparation, built strictly from closed-session data — reviewed in calm, when no regular-session order could execute anyway.

Stack decisions in detail

Each layer: what was chosen, why, what else was considered, and what you give up. The bias throughout: simplicity and low ops burden beat sophistication for a single-user system.

3.1 Frontend — Next.js App Router + TypeScript + Tailwind + shadcn/ui

Why: your core skill set; server components read the serving tables via Prisma with zero client data-fetching complexity, which suits a read-mostly app perfectly. **Notes:** chart components are client-side (`'use client'` + `useEffect`), or `next/dynamic` with `ssr: false`) since every serious chart library is canvas-imperative; Next.js 16 renamed middleware to `proxy` — the Basic Auth gate lives there. The app writes only user-state tables (watchlist, journal, review state, paper trades, weakener checkboxes) via server actions; everything pipeline-derived is read-only to it. **Alternatives:** none seriously — nothing beats the incumbent skill. **Tradeoffs:** heavy on-demand computation must go to the pipeline (or a manually triggered `workflow_dispatch`), never a Vercel function — the Hobby 300-second cap is a hard wall.

3.2 Serving database — Supabase free Postgres + Prisma

Why: you already know both. Prisma owns the schema and migrations; the Python pipeline INSERTs via a connection string from GitHub secrets. The feared 7-day inactivity pause is moot while the pipeline writes nightly. **Connection fine print (matters in CI):** three URLs, not two — the pooled transaction-mode `DATABASE_URL` for the app and the pipeline's normal writes; `DIRECT_URL` for migrations run from your machine; and the **Supavisor session-mode pooler string (IPv4, port 5432)** for anything running on GitHub-hosted runners (`pg_dump` , CI migrations) — Supabase free-tier direct connections are IPv6-only and GitHub runners have no IPv6. **Alternative:** Neon free (100 CU-hours/mo, native Vercel Marketplace) is equally valid and decouples database health from pipeline health — with Prisma it is a connection-string change. **Tradeoffs:** a pipeline broken for 7+ days pauses the Supabase project (manual dashboard restore) — mitigated by the dead-man alert, which surfaces a broken pipeline the same evening (~9:10pm EDT / 8:10pm EST). The 500MB ceiling means full price history must not live here — hence:

3.3 History store — Parquet on Cloudflare R2, queried by DuckDB

Universe, defined once and used everywhere: US common stocks and ETFs listed on NYSE, Nasdaq, and AMEX (no OTC), ~5–6k active symbols, with delisted symbols retained for survivorship honesty. At Alpaca's free 7+ years of history that is $\sim 5.5k \times 252 \times 7 \approx 9.7M$ rows $\approx 120\text{--}350\text{MB}$ of Parquet — DuckDB full-scans that in seconds; Stooq/ yfinance backfill can extend the archive toward 10 years later if base rates want deeper samples. Postgres adds schema ceremony for zero benefit at an append-nightly/scan-nightly access pattern. R2 is deliberately *a cache, not a system of record*: Alpaca history is re-downloadable, so losing the bucket costs a re-pull, not data. **Adjustment correctness:** raw OHLCV is stored alongside adjusted values; on any corporate action the affected symbol's rows are rewritten across *all* year partitions — “immutable prior years” describes the layout's default, never a bar to adjustment corrections. **Alternatives considered:** everything-in-Postgres (blows the free tier), GitHub Actions cache (7-day eviction — wrong tool), committing

Parquet to the repo (~1GB/month of git bloat). **Tradeoffs:** one extra account and credential. R2's free tier (10GB per research) was *not* independently verified — confirm at signup; Backblaze B2 is the fallback.

3.4 Pipeline runtime — Python 3.12: Polars + DuckDB + exchange_calendars + httpx

Why: the consensus 2026 pattern for exactly this workload — Polars for ingest/transform with lazy, vectorized expressions; DuckDB as the SQL engine over Parquet for “indicator state → forward return” window queries (zero-copy interop between them). **Indicators are hand-rolled** (~200 lines of Polars expressions for SMA/EMA/RSI/ MACD/ATR/Bollinger and friends): pandas-ta passed its announced July 2026 archival deadline, and TA-Lib's C dependency is CI install pain you don't need for daily-bar math. Unit-test each indicator against known reference values — this fits your TDD workflow and you own correctness. Base rates are a `group_by` + shift — ~50 lines of custom code, not a backtesting framework. **Rerun semantics are designed, not hoped:** every serving write carries a natural key with `ON CONFLICT DO NOTHING` (signals) or full-refresh semantics (scans), per-stage completion rows in `pipeline_run` let a `workflow_dispatch` rerun skip completed stages, and the whole night is safe to run twice. **Local development:** each provider adapter ships with recorded fixture responses; tests run against fixtures + a docker-compose Postgres + a local MinIO (or dev R2 bucket); CI runs the same fixtures — no test ever touches a live API key. **Alternatives:** TA-Lib (fine, actively maintained, use it if you'd rather not own indicator correctness); vectorbt free edition (reserve for optional parameter sweeps); pandas-ta-classic fork (longevity risk). **Compute reality:** the full-universe scan is minutes on a 2-vCPU runner; wall-clock is dominated by provider APIs, not math.

3.5 Scheduler — GitHub Actions, and why not Vercel

The forced departure. Verified against official docs (July 2026): Vercel Hobby caps *every* function — cron included — at 300 seconds with no extension; daily Hobby crons fire at an arbitrary minute within the hour; and pandas/numpy-sized Python bundles fight the packaging limits. Vercel Pro (\$20/mo) buys 800s functions — worse Python ergonomics at four times the price of a VPS, rejected. A private-repo GitHub Actions workflow gives a full Python environment, a 6-hour job limit, ~2,000 free minutes/month, and `workflow_dispatch` for manual reruns. **The minutes math is why the night is two jobs:** a single job that blocked on the ~35–40-minute LLM batch would bill ~78 minutes/night ≈ 1,650/month — uncomfortably close to the free cap, and exhausting it mid-month is precisely the silent death the design fears. Splitting at batch submission (Job A ~40 min + Job B ~10 min ≈ 50 billed minutes/night ≈ **1,050/month**) leaves ~45% headroom for reruns, backfills, and the weekly backup step. Cron jitter (5–30 min, community-documented) is irrelevant inside the after-close slack; the `:37/ :25` odd minutes dodge top-of-hour congestion. **Reliability fine print:** GitHub auto-disables schedules in repos with no activity for 60 days — documented for *public* repositories; whether it applies to private repos is unverified, so a zero-cost heartbeat step stays in the workflow and the dead-man ping remains the primary defense. Schedule events can also be delayed or occasionally *dropped* under platform load — `workflow_dispatch` is the manual recovery, and the dead-man alert is what tells you to use it. **Escape hatch:** a Hetzner CX22 VPS (~\$5/mo, unverified — confirm at checkout) with a systemd timer, if exact timing or >6h runs ever matter; the DB stays managed so the box remains disposable.

3.6 Charts — TradingView Lightweight Charts v5, and the licensing trap avoided

Why: a purpose-built canvas candlestick engine at 59KB gzipped, Apache-2.0 with an attribution requirement satisfied by the built-in TradingView logo (on by default — leave it on and you are compliant for free). v5 brings panes (RSI/volume sub-panes), series markers and price lines — exactly the primitives the explain-why annotations need — plus a custom-primitives plugin API if annotations get fancy. Integrate via the MIT React wrapper (`lightweight-charts-react-components`, React 19-verified) or a ~30-line hand-rolled hook (TradingView’s own React example; zero dependency risk — arguably the right call here). **The trap:** TradingView *Advanced Charts*’ free license is companies-only, explicitly excluding personal/hobby use — legally unavailable to this project regardless of its feature appeal. **Fallback:** Highcharts Stock is free for personal non-commercial use (CC BY-NC) and is the turnkey option if you ever want a built-in range-selector/drawing UI; ApexCharts is no longer MIT and its SVG renderer struggles at candlestick data volumes; chartjs-chart-financial is dormant. **General viz:** Recharts 3 (the shadcn/ui charts foundation) for sparklines and small multiples; add tree-shaken ECharts 6 via `echarts/core` only when the sector heatmap becomes real.

3.7 Repository layout

```
myStockMarket/
├─ app/                                # Next.js (Vercel project root)
│  ├─ app/(desk)/...                  # Desk routes: dashboard, ticker/[symbol], track-record
│  ├─ app/(academy)/...               # Academy routes: modules, lessons, glossary, review
│  ├─ app/api/revalidate/             # CRON_SECRET-gated revalidation
│  ├─ proxy.ts                        # Basic Auth (Next 16 middleware rename)
│  └─ prisma/schema.prisma           # owns the serving schema
├─ pipeline/                          # Python 3.12 (uv-managed)
│  └─ adapters/                      # alpaca.py · finnhub.py · fmp.py · edgar.py · fred.py ·
marketaux.py                         # + fixtures/ (recorded responses for tests & local dev)
│  │
│  ├─ indicators.py                  # ~200 lines of Polars expressions + tests
│  ├─ scans.py · baserates.py · volbands.py · briefing/ (extract.py · synthesize.py · verify.py)
│  ├─ publish.py                    # single-transaction refresh into Supabase
│  └─ tests/                        # indicator reference values, base-rate lookahead guards, Wilson CI
cases
└─ .github/workflows/
   ├─ nightly-a.yml                 # cron 37 22 * * 1-5 – ingest, compute, submit batch (+ heartbeat)
   └─ nightly-b.yml                 # cron 25 0 * * 2-6 – collect, synthesize, verify, publish (+ weekly
pg_dump)
```

One repo, two deployables, two short workflows. The `adapters/` directory is the repository pattern the data plan depends on: every provider is swappable in one module, because free tiers have a documented habit of being cut (Alpha Vantage 500→25 req/day; Finnhub’s free candles removed).

The data plan

A \$0 baseline with confirmed US coverage — NYSE/Nasdaq breadth, the US earnings calendar, US filings and macro — verified against official pricing pages on 8–9 July 2026, plus a pre-researched upgrade ladder so the response to free-tier friction is a same-week migration, not a scramble.

4.1 The \$0 baseline

CHOSEN SOURCE PER DATA TYPE — ALL FREE, ALL VERIFIED AGAINST OFFICIAL PAGES, ALL US-COVERAGE-CONFIRMED

Data type	Provider	Free-tier terms that matter	The gotcha to respect
Full-universe EOD bars + 7y history (NYSE/Nasdaq/AMEX)	Alpaca Market Data Basic ● VERIFIED	\$0 · 200 API calls/min · multi-symbol bars endpoints · corporate actions included · no brokerage account needed · “100% market coverage”.	Historical REST excludes the latest 15 minutes — irrelevant for a post-close batch. Free quotes are IEX-only (~2–3% of consolidated US volume); real-time consolidated SIP is Alpaca’s \$99/mo tier — irrelevant for an EOD-centric v1, but the number to know.
Fundamentals (metrics, profiles)	Finnhub free ● VERIFIED	\$0 · 60 calls/min · <code>/stock/metric</code> confirmed non-premium · US coverage.	OHLCV candles are premium-only on Finnhub — never plan price data here; free keys get 403 on <code>/stock/candle</code> .
US earnings calendar (bmo/amc timing)	FMP free ● VERIFIED	\$0 · 250 calls/day · 500MB/30-day bandwidth — ample for one nightly pull of the US calendar.	End-of-day timeframe only on free; 5-year history cap; free-tier scope is US companies.
Filings (8-K, 10-Q/K) for watchlist	SEC EDGAR ● VERIFIED	\$0 forever, public domain, the primary source for US filings · per-CIK submissions JSON, full-text search, nightly bulk ZIPs · real-time dissemination.	10 req/s cap and a mandatory <code>User-Agent: "Name email@domain"</code> header — undeclared tools get 403’d.
US macro series + release calendar (CPI, jobs, rates)	FRED ● VERIFIED	\$0 with free API key · 845k series · v2 bulk API (Nov 2025) · release-dates endpoints cover the US macro calendar.	2 req/s documented limit; mandatory attribution notice must render in the app footer.
Ticker-tagged news + sentiment	Finnhub company-news + Marketaux ● PARTIALLY VERIFIED	Finnhub news free for watchlist scope; Marketaux free = 100 req/day × 3 articles ≈ 300 snippets/night with per-entity sentiment — the market-wide catalyst source for non-watchlist movers (partial coverage; unmatched movers honestly render “no news found”).	Marketaux returns snippets only, never full text, on all plans; archive depth undocumented.

Data type	Provider	Free-tier terms that matter	The gotcha to respect
Backfill / cross-check only	yfinance · Stooq ● FALLBACK TIER	\$0 · full-market EOD + decades of US history — the path to extending the archive toward 10 years.	yfinance is an unofficial scraper: personal-use-only per Yahoo ToS, undocumented rate limits, breaks recurrently. Never a foundation — a monthly cross-check and emergency backfill.
Deferred: live watchlist quotes	Alpaca IEX WebSocket ● POST-V1	\$0, real-time, 30-symbol cap — the documented path <i>if</i> an evening live view is ever wanted.	Deliberately not in v1: the app is as-of-close by design, and provider keys stay server-side — a live view would require a relay the architecture doesn't otherwise need. Revisit only with a stated use case.

LICENSING — ONE RULE COVERS EVERYTHING

Every shortlisted provider licenses data for **personal, non-redistributed use with local storage**; several (Finnhub, Tiingo most strictly) forbid showing data to anyone else, and even paid self-serve tiers stay personal-use. US real-time redistribution is further bound by exchange agreements (SIP/OPRA) at the paid tiers — another reason v1 stays end-of-day. The always-on Basic Auth wall is the compliance mechanism. Keep the FRED attribution line and the TradingView logo rendered; no public deploy previews of data pages (Vercel's free preview authentication covers this).

4.2 The upgrade ladder — pay only when a named friction appears

PRE-RESEARCHED PAID STEPS, WITH VERIFIED PRICES AND EXPLICIT TRIGGERS

Upgrade	Price (verified 8–9 Jul 2026)	Adopt when...	Fine print that matters
EODHD "EOD Historical Data" (All World)	\$19.99/mo	The Alpaca nightly batch shows friction: long-tail ticker gaps, split/adjustment issues, or rate-limit pain. Likely the first upgrade.	Bulk endpoint returns an entire exchange's EOD for 100 weighted calls (of 100k/day). Watch call-weighting: fundamentals cost 10 calls, intraday/news 5. Free tier (20 calls/day) is demo-grade.
FMP Starter	\$22/mo billed annually	The free 250 calls/day binds — i.e., nightly fundamentals/news for more than a handful of tickers.	Month-to-month price is higher and unverified (~\$29–33 plausible — do not budget on it). Starter = 5y history, <i>annual</i> statements only, US coverage; quarterly/TTM needs Premium \$59/mo annual-billed.

Upgrade	Price (verified 8–9 Jul 2026)	Adopt when...	Fine print that matters
Tiingo Power	\$30/mo or \$300/yr	Adjusted-close data quality becomes the complaint — Tiingo has the best cleanliness reputation in this class, with 30+ years of US EOD history.	Fundamentals is a separately priced add-on (unlisted). License is strictly personal/internal; the \$50/mo tier is for internal commercial use.
Massive (ex-Polygon) Stocks Starter	\$29/mo or \$288/yr	Developer experience and unlimited API calls matter more than freshness: grouped-daily endpoint = the whole US market in one call, best-in-class docs.	15-minute delayed (fine here). Real-time needs Advanced at \$199/mo with non-professional exchange agreements — don't. Note the rebrand: polygon.io now redirects to massive.com.
Marketaux Basic	\$29/mo (\$24 annual)	300 snippets/night feels thin for market-wide mover coverage: jumps to 2,500 req/day × 20 articles.	Still snippets-only; ToS is personal/non-commercial boilerplate.
Finnhub Fundamental-1	\$50/mo, billed quarterly	Only if you want standardized + as-reported statements under a key you already integrate.	Per-market licensing; strictly personal-use even when paid; no refunds; 30-day cancellation notice. FMP Premium is usually the better fundamentals dollar.
Hetzner CX22 VPS (ops, not data)	~\$5/mo ● UNVERIFIED	GH Actions jitter, dropped schedule events, or the 6h cap ever bite: exact-minute systemd timer, unlimited duration, persistent Parquet disk.	Third-party trackers cite €4.49–4.59 after an April 2026 price adjustment — confirm at checkout. DB stays managed; the box stays disposable.

Upgrade philosophy: consolidate onto *one* paid provider when friction appears rather than stacking several — the likely first move is EODHD (EOD breadth) or FMP Starter (fundamentals breadth), whichever pain arrives first. **Evaluated and rejected:** Alpha Vantage (25 req/day free — precedent of cutting free tiers), Twelve Data, Marketstack, Databento, NewsAPI.org, Benzinga (enterprise pricing), Trading Economics, and scraping Nasdaq/Zacks (legality). Every adapter sits behind the repository interface, so any of these verdicts can be reversed in one module. Account setup is unremarkable for a US-resident signup — create the four keyed accounts (Alpaca, Finnhub, FMP, Marketaux) at kickoff and smoke-test each before writing adapters; per-provider fallbacks stay in the ladder for outages and term changes, and EDGAR/FRED need no account at all.

The AI layer

Three stages, five grounding rules, one hard gate. The LLM reads and writes; it never counts, never predicts, and never publishes unverified numbers.

5.1 Extract → synthesize → verify

Stage	Design	Model class · run mode · cost/night
A · Per-article extraction	Each news item (~100/night from Finnhub + Marketaux) → rigid JSON: 250-token summary, tickers, sentiment, event type, mandatory doc-ID citation . Prompt forbids outside knowledge; the model sees only the fetched article text. Structured outputs enforce the schema.	Cheap-fast (Haiku-class: <code>claude-haiku-4-5</code>) · Batch API, -50% — submitted by Job A, collected by Job B ~65 min later ~\$0.35 list / ~\$0.18 batched
B · Briefing synthesis	One call composes the evening briefing — what moved in the US session, why (per cited extracts), what the scanner found, what's on tomorrow's US calendar. It sees only the structured extracts plus the pipeline's computed-stats table; every number used verbatim; every bullet carries a source ID, enforced by output schema.	Frontier (Sonnet-class: <code>claude-sonnet-5</code>); Opus-class adds ~\$0.08/night if prose quality ever warrants) · synchronous in Job B — one ~90-second call; batching one call buys nothing and would chain a second batch wait ~\$0.13
C · Verification gate	Deterministic, <i>not</i> an LLM judge — with defined tolerances, not naive string equality: numbers are parsed to values and matched within rounding tolerance (a drafted "8.2%" matches a sourced 8.24%), tickers and dates canonicalized, units normalized (\$1.2B ≡ 1,200M). Every number, ticker, and date in the draft must match the source extracts or the stats table; violations are flagged inline or block publish. (LLM citation-validation runs ~38% accuracy in studies; deterministic checking is near-free.) Optional cheap-model spot-check on non-numeric claims.	Parser/regex + optional Haiku pass · synchronous ~\$0–0.03

5.2 The five grounding rules

1. **The pipeline computes all numbers; the LLM narrates them verbatim.** Benchmark evals show top models collapsing from ~95% on simple financial lookups to near 0% on multivariate calculation — so arithmetic is simply not the model's job here.

- 2. **Ground on fetched documents only:** every input carries a doc ID; outside knowledge is forbidden; every claim must cite a doc ID — enforced via structured output schema (`output_config.format` / `client.messages.parse`), not via hope.
- 3. **Assume residual hallucination and gate it.** Grounded-summarization hallucination rates are ~0.7–1.5% even for top models — low, not zero. Stage C exists to catch the residue; citations render inline in the UI so the user can always click through.
- 4. **Two-stage architecture caps the blast radius:** the synthesis model never sees raw articles — only schema-validated extracts — bounding both context size and hallucination surface.
- 5. **No verbalized probabilities, ever.** LLM-stated confidences are systematically overconfident; every probability on screen is a frequency computed from historical data by the pipeline (Research Report, Part 7).

5.3 Cost math, shown in full

NIGHTLY TOKEN ARITHMETIC — API PRICES AS OF THE OFFICIAL PRICING CACHE, JUNE 2026
(RECONFIRM AT BUILD TIME)

Stage	Input tokens	Output tokens	Rate (per MTok)	\$/night
A · Extraction — 100 items × (≈1,200 article + 1,000 prompt/schema in; 250 out) · batched –50%	220K	25K	Haiku 4.5 · \$1 / \$5	\$0.18
B · Synthesis — extracts (25K) + 5K stats + 2K instructions; ~2K out · sync	32K	2K	Sonnet 5 · \$3 / \$15 intro \$2/\$10 through Aug 31, 2026	\$0.13
C · Optional spot-check · sync	27K	0.5K	Haiku 4.5	\$0.03
Typical night				≈ \$0.33
All-sync fallback night (batch cancelled at cutoff; already-processed batch items still bill)				≈ \$0.51
Monthly (~21 US trading days)			≈\$7 typical · ≈\$11 if the fallback fired every night	≈ \$7– 11

Only Stage A is batched — it is where the volume is, and the Batch API is the right 50% discount for work nobody is waiting on. Stage B is a single call whose input depends on Stage A’s output; batching it would chain a second batch wait for a ~\$0.07 saving — not worth it. Prompt caching is *not* the lever here either: the shared prefix (~1K tokens) sits below the minimum cacheable prefix for Haiku-class models (2–4K tokens depending on model), and the hundred calls differ in body anyway. Local models were evaluated and rejected: a CPU-only runner is slower and measurably worse, and 2026 TCO analyses put self-hosting far above a ~\$7–11/month API bill.

WHAT THE LLM NEVER DOES

No arithmetic. No probabilities. No direction. No “I think this stock will...”. No content that lacks a source ID. If Stage C finds an unsourced number, the briefing publishes with that line flagged — or not at all: **data beats prose**, and a “briefing unavailable tonight” banner over correct scan results is an acceptable outcome; a fabricated number read before the open is not.

Data model

A deliberate split: full-market history lives as Parquet files the pipeline scans; Postgres holds what the app serves — plus the small set of tables the user writes (watchlist, journal, review state, paper trades). Two stores, each doing the one thing it is good at.

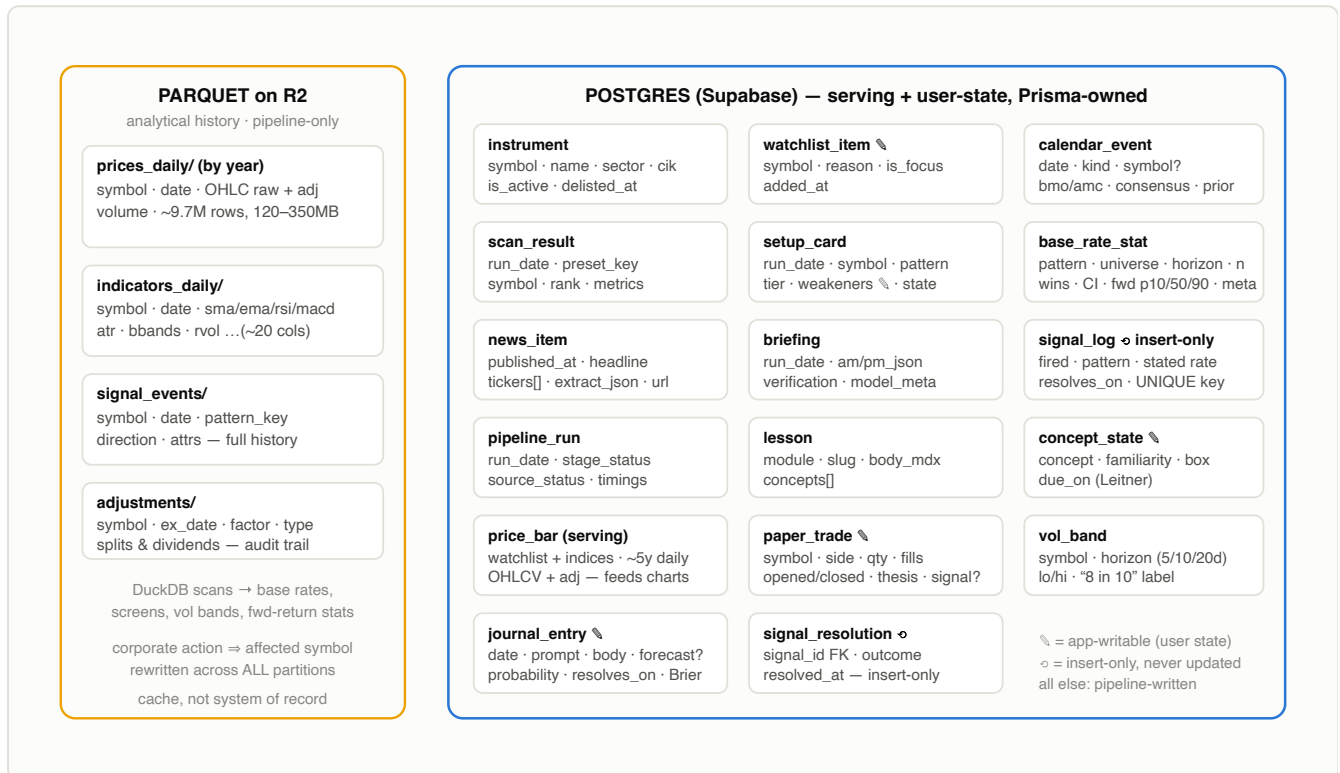


Figure 6.1 — Data model zones. Pipeline-derived tables are written only by the nightly jobs; the app writes only the 🔗 user-state tables. `setup_card` references `base_rate_stat`; every card also inserts a `signal_log` row at fire time, and the resolver later inserts a matching `signal_resolution` row — the log itself is never updated. Instrument relations (symbol foreign keys) omitted from the drawing for legibility.

6.1 Table definitions — the fields that carry design decisions

Table	Purpose & the columns that matter
instrument	Symbol master for the defined US universe (NYSE/Nasdaq/AMEX). <code>is_active</code> / <code>delisted_at</code> exist so backtests can include dead tickers — the survivorship guardrail from the Research Report is a schema feature, not a hope.
price_bar	The serving copy of price data — watchlist plus index/sector symbols × ~5 years of daily bars (~40–60k rows, trivial inside the 500MB ceiling), written nightly like every serving table. Candlestick charts and sparklines render from here; full-market history stays Parquet-only, and the app never needs R2 credentials.
base_rate_stat	One row per (pattern_key, universe, horizon, regime): <code>n</code> , <code>wins</code> , <code>win_rate</code> , <code>ci_low</code> , <code>ci_high</code> (Wilson), <code>fwd_p10/median/p90</code> , plus pattern metadata: <code>publication_year</code> , <code>evidence_grade</code> (the Part-4 ledger verdict), and <code>decay_note</code> — the decay stamp is a schema field, not a caption. The UI's display-precision rules key off <code>n</code> ; the tier is capped at "weak" whenever the CI includes the always-up baseline.

Table	Purpose & the columns that matter
vol_band	The product's only sanctioned forecast: per-watchlist-symbol expected-range bands at 5/10/20 trading days from empirical quantiles of recent realized moves (HAR-lite), each row carrying its frequency label ("8 in 10 past paths stayed inside") and the regime-break caveat string. Nothing beyond 20 days exists, by schema.
setup_card	A day's rendered signal: pattern state, tier, weakeners with live check-state (the checkbox is user-writable), FK to its base-rate row. Cards are regenerated nightly; history lives in the log.
signal_log + signal_resolution	The track record, mechanically honest. <code>signal_log</code> is insert-only (UPDATE/DELETE revoked): fired signal + the exact numbers stated at fire time + <code>resolves_on</code> , with <code>UNIQUE (fired_date, pattern_key, symbol, horizon)</code> and <code>ON CONFLICT DO NOTHING</code> so reruns are idempotent. The nightly resolver <i>inserts</i> a <code>signal_resolution</code> row (outcome, resolved_at) — outcomes live in their own insert-only table, so append-only is a mechanical property, not a convention. Logging starts in Phase 1, resolution in Phase 4.
paper_trade	The paper-trading ledger — v1's only "orders." Symbol, side, qty, simulated fill (next-open price + half-spread + slippage estimate), opened/closed timestamps, a required thesis note, optional link to the signal that prompted it. Turnover from this ledger drives the cost mirror and the trade-frequency mirror; a paper order placed within minutes of a signal firing triggers the cooling-off interstitial. This table is what makes "paper-first" a feature rather than a slogan.
briefing	AM/PM JSON plus <code>verification_json</code> (every extracted number/ticker/date and where it matched) and <code>model_meta</code> (models, token counts, batch vs sync) — the audit trail for every published word.
news_item	Snippet + provider URL + the Stage-A extract. Snippets only (Marketaux never returns full text); the UI always links out.
calendar_event	US earnings (with bmo/amc), US macro prints (consensus/prior — CPI, jobs report, GDP), FOMC dates and Fed speakers. The if/then scenario cards render from this plus <code>base_rate_stat</code> — and a branch whose historical sample has $N < 30$ renders "insufficient history for this branch — treat as anecdote" rather than a number.
pipeline_run	Per-stage and per-source status — surfaces as the dashboard footer ("news unavailable tonight") so degraded runs are visible, never silent. Stage-completion rows are what make <code>workflow_dispatch</code> reruns skip finished work.
lesson / concept_state	Academy content (MDX) and the Leitner review queue (<code>box</code> , <code>due_on</code> , familiarity for adaptive fading). <code>concept_state</code> is app-written on every review answer.
journal_entry	PM-prompt responses — plus the forecast-journal fields the Research Report commits to: optional <code>forecast</code> (a falsifiable statement), <code>probability</code> (anchored on a displayed base rate), <code>resolves_on</code> , <code>outcome</code> , so user predictions are Brier-scored on the same track-record page as the app's own flags. Together with <code>paper_trade</code> , the only truly irreplaceable data in the system — see backup policy, §7.3.

Parquet layout: `prices_daily/year=YYYY/*.parquet` (current year rewritten nightly; prior years rewritten per-symbol when a corporate action lands), `adjustments/` as the split/dividend audit trail, `indicators_daily/` and `signal_events/` regenerated from prices as needed. Losing the bucket costs one re-pull from Alpaca — by design, nothing in R2 is precious.

Security, operations & cost

Single-user security is mostly about not inventing problems: one wall, few secrets, and monitoring designed around the one catastrophic failure mode — silence.

7.1 Authentication & the licensing wall

HTTP Basic Auth implemented in ~20 lines of Next.js middleware (`proxy.ts` in Next 16): credentials in env vars, constant-time comparison, HTTPS via Vercel. This is deliberately boring — for one user it is exactly sufficient, and it doubles as the data-licensing compliance mechanism (every provider requires the dashboard stay private to you). The pipeline’s revalidate route requires an `Authorization: Bearer $CRON_SECRET` header. Vercel’s free preview-deployment authentication stays on so preview URLs never expose data pages. Rejected: Vercel Password Protection (requires a \$150/mo add-on — absurd here), Clerk (dependency overkill for $n=1$); the free step-up if Basic Auth’s browser popup ever grates is Auth.js with a single credentials user, or Cloudflare Access in front of a custom domain.

7.2 Secrets inventory

Secret	Lives in	Notes
<code>DATABASE_URL</code> (pooled, transaction mode)	Vercel env + GH Actions secrets	App reads and user-state writes; pipeline bulk writes.
<code>DIRECT_URL</code> / <code>SESSION_POOLER_URL</code>	Local <code>.env</code> / GH Actions secrets	<code>DIRECT_URL</code> for migrations from your machine. The Supervisor <code>session-mode</code> string (IPv4, port 5432) is what CI uses for <code>pg_dump</code> and migrations — Supabase free direct connections are IPv6-only and GitHub runners have no IPv6.
<code>ALPACA_KEY_ID</code> / <code>ALPACA_SECRET</code> , <code>FINNHUB_KEY</code> , <code>FMP_KEY</code> , <code>MARKETAUX_KEY</code> , <code>FRED_KEY</code>	GH Actions secrets only	Provider keys never reach Vercel or the client. EDGAR needs no key — only the declared User-Agent.
<code>ANTHROPIC_API_KEY</code>	GH Actions secrets only	Set a monthly spend limit in the console as a belt-and-suspenders cost cap.
<code>R2_ACCESS_KEY_ID</code> / <code>R2_SECRET</code>	GH Actions secrets only	Scoped token, one bucket. The app never touches R2 — charts read <code>price_bar</code> in Postgres.
<code>BASIC_AUTH_USER</code> / <code>BASIC_AUTH_PASS</code> , <code>CRON_SECRET</code>	Vercel env	Rotate by redeploy; no code change.
<code>HEALTHCHECKS_PING_URL</code>	GH Actions secrets	Treat as a secret — anyone with it can fake liveness.

Nothing uses `NEXT_PUBLIC_`. The repo is private; secrets exist only in GitHub Actions encrypted secrets and Vercel env vars, pulled locally via `vercel env pull` when needed. Local development and CI never use live keys — adapters run against recorded fixtures (§3.4).

7.3 Monitoring, failure design, and backups

- **The catastrophic failure is a silently dead cron** — stale data quietly served day after day. Defense in depth: a zero-cost heartbeat step in the workflow (the 60-day schedule auto-disable is documented for public repos; unverified for private — the heartbeat removes the question either way); workflow-failure e-mails catch crashes; and a **healthchecks.io dead-man check configured on the pipeline’s own cron schedule** (`25 0 * * 2-6`, ~45-minute grace) so weekends never false-alarm — absence of success alerts by ~9:10pm EDT (8:10pm EST), the same evening, not days later. The holiday-exit path pings success explicitly so US market holidays don’t false-alarm either.
- **Degraded > dead:** each data source is isolated, so one provider outage turns into one “unavailable tonight” section, recorded in `pipeline_run` and shown in the dashboard footer. Ingest hard-fails if <95% of the expected universe returns, so stale data never masquerades as fresh.
- **Publish is one transaction:** all serving tables are refreshed inside a single transaction (DELETE-and-INSERT of the night’s serving rows, or staged tables renamed together in the same transaction) — MVCC serves readers the previous consistent snapshot until commit, so the Desk never renders a half-written briefing and foreign keys never dangle across generations.
- **Backups follow value:** Parquet is re-pullable and pipeline-derived Postgres tables are regenerable — but `signal_log`/`signal_resolution`, `paper_trade`, and `journal_entry` are irreplaceable, so a weekly `pg_dump` step (over the session-mode pooler — see §7.2) ships a compressed dump to R2. Restore is one command; testing it once is a Phase 1 acceptance criterion.

7.4 The monthly bill

Scenario	\$/month	Composition
Low	~\$7	Batch discount lands every night ($\$0.33 \times 21$ trading days $\approx$ \$6.90); all free tiers behave.
Expected	\$7–11	Entirely LLM tokens: occasional all-sync fallback nights (~\$0.51), reruns, and development/testing tokens. \$11 is the ceiling even if the batch discount never lands; an Opus-class synthesis upgrade would add ~\$1.70/mo on top. Everything else — hosting, DB, scheduler, storage, charts, auth, monitoring — rides verified free tiers.
High (upgrades fired)	~\$65–105	EODHD EOD+Intraday \$29.99 + FMP Premium \$59 (annual-billed) + Hetzner ~\$5 + ~\$11 LLM — adopted one at a time on named triggers, never all at once.

Implementation roadmap

Seven phases, each shippable and each delivering daily value before the next begins. Three sequencing decisions are deliberate: the signal log starts recording in Phase 1 (so the track record is long by the time its page ships); setup cards never ship before their accountability surface (the resolver and a visible resolved log land in the same phase); and paper trading arrives after the Academy exists, gated by the risk module — watch first, learn risk, then size paper positions.

PHASES WITH ACCEPTANCE CRITERIA — “DONE” IS TESTABLE, NOT VIBES

Phase · effort	Scope	Acceptance criteria
P0 · Walking skeleton 1 weekend	Repo scaffold (app/ + pipeline/ + two workflows); Next.js deployed on Vercel behind Basic Auth; Supabase + Prisma migrated; Job A cron runs a hello-pipeline that writes one <code>pipeline_run</code> row; the page renders it. Dead-man ping wired from day one. All provider free-tier keys created and smoke-tested before any adapter is written.	Open the app and see the evening run's timestamp, produced entirely by the cloud schedule — nothing ran on your hardware. Kill the workflow once → healthchecks alert arrives the same evening. Alpaca, Finnhub, FMP, Marketaux, FRED keys all live in GH secrets.
P1 · Data spine 1–2 weeks of evenings	Alpaca adapter → full-universe EOD → Parquet on R2; indicator library (Polars, unit-tested against reference values); DuckDB scan queries; <code>instrument</code> , <code>price_bar</code> , <code>scan_result</code> , and <code>signal_log</code> tables live; minimal Desk: US macro pulse (index ETFs + VIX via FRED), movers table with RVOL, watchlist CRUD, slide-over rail (click any ticker → in-place rail), ticker page with Lightweight Charts candles fed from <code>price_bar</code> . Weekly <code>pg_dump</code> step over the session pooler.	Nightly Job A ingests the universe in <30 min; indicator tests green; ingest hard-fails below the 95% threshold (tested with a faked partial response); designated presets write <code>signal_log</code> rows with <code>resolves_on</code> set and rate fields null (until P4); rail opens without route change and Esc restores scroll + selection; backup restore tested once.
P2 · Catalyst & context ~1 week	Remaining adapters: FMP US earnings calendar (bmo/amc), Finnhub news + metrics, Marketaux (the market-wide catalyst source), EDGAR submissions, FRED releases. Movers-with-reason (catalyst matched by ticker + time window); US session calendar (earnings, CPI, jobs report, FOMC) — if/then cards render event + consensus/prior only, with per-branch base rates attaching in P4; per-source status footer.	Every >3% mover on the movers table shows a catalyst chip with source link or the honest “no news found — likely noise” tag (market-wide coverage is partial by design and says so); killing one provider key degrades one section, not the run.
P3 · The briefing ~1 week	LLM pipeline split across the two jobs: Job A submits the extraction batch; Job B collects, runs the one synchronous synthesis call, the deterministic verification gate (tolerance rules from §5.1), and the single-transaction publish; <code>briefing</code> table with verification audit; AM brief + PM scorecard rendering (scorecard displays “grading begins in P4” until the resolver exists); “briefing unavailable” degradation banner.	Five consecutive nights of briefings in which every number/ticker/date passes the gate; a seeded fake number in a test draft is caught and blocked; the batch-cutoff fallback drill exercised once; publish confirmed atomic (no mixed-generation reads under a mid-publish request).

Phase · effort	Scope	Acceptance criteria
P4 · Setup cards, base rates & accountability ~2 weeks	First pattern detectors (MA cross, 52-week-high proximity, gap-with-catalyst, RSI extreme, unusual volume, breadth states) writing <code>signal_events</code> history; <code>base_rate_stat</code> with Wilson CIs, regime slices, and the decay-stamp metadata (publication_year, evidence_grade, decay_note); <code>vol_band</code> computation (5/10/20-day empirical quantiles) and fan-chart render; lottery-risk flags (high-skew, low-price) computed in the scan stage; setup-card UI with icon array, reference-class chip, weakeners, baseline-comparison line; display-precision rules; evidence-grade labels and visible criteria on scan presets; the nightly resolver (<code>signal_resolution</code>) and a minimal track-record page (plain resolved log) — cards do not ship without them; if/then branch base rates attach, with the N<30 suppression rendering.	Every card shows N + CI + the unconditional-baseline line; a CI spanning the always-up baseline renders tier WEAK (unit-tested); an N<30 pattern renders “insufficient history” with the percentage suppressed; the one-bar-shift lookahead guard has a failing-test proof; every fired card logs at display time; the resolved log shows outcomes within a day of each horizon passing; no vol band renders beyond 20 trading days.
P5 · Academy 2–3 weeks (content-heavy)	MDX lesson framework; ~25 launch lessons across M0–M6 (grow toward ~60), M1 anchored on US market mechanics and session structure; glossary popovers (first-occurrence discipline); worked-example drawer with on-chart annotations; Leitner review queue (max 5/day); bidirectional links with return rails; risk-module soft gate.	Every term on the Desk has a glossary entry; drawer → lesson → “Return” restores Desk state exactly; a pattern lesson opened before M3 completion shows the soft gate; review queue never exceeds 5 items; Academy-internal breadcrumbs (map → lesson) present.
P6 · Paper desk & calibration ~2 weeks + ongoing	The <code>paper_trade</code> ledger and entry UI (simulated next-open fills + half-spread + slippage), soft-gated on Academy M3; cost mirror computed from the user’s <i>actual</i> paper turnover; cooling-off interstitial (paper order within X minutes of a signal firing) and weekly trade-frequency mirror — the behavior-mirror push channel of RR §6.2, rate-limited; forecast-journal fields on <code>journal_entry</code> with Brier resolution; half-Kelly-capped paper sizing helper (CI-lower-bound win rate + stored payoff percentiles); full track-record page (calibration curve with per-bucket N, rolling Brier with the “0.25 = coin flip” anchor) now covering both the app’s flags and the user’s forecasts; PM scorecard auto-grading; quantile dotplots in drill-down; sector small multiples; dark mode (Desk only — Academy stays light); ⌘K.	Track-record page shows ≥30 resolved signals including misses; cost mirror reproduces the spread×turnover arithmetic on the user’s ledger (fixture ledger in tests); a paper order placed <X min after a signal fires triggers the cooling-off interstitial (tested); sizing helper never exceeds half-Kelly (property test); user forecasts resolve and Brier-score alongside app flags.

TRACEABILITY — RESEARCH REPORT WIDGET CATALOG → BUILD PHASE

Widget (RR §9.2)	Priority	Phase
US macro pulse + breadth strip	Must	P1 (breadth internals mature in P4)
Movers with reason	Must	P1 (table + RVOL) → P2 (catalyst chips)
Focus watchlist + slide-over rail	Must	P1
US session calendar + if/then cards	Must	P2 (events) → P4 (branch base rates)
Daily brief pair (AM/PM)	Must	P3 (PM auto-grading P4–P6)
Probabilistic setup cards	Must	P4 (with resolver + minimal track record)

Widget (RR §9.2)	Priority	Phase
Track-record page	Must	P4 minimal → P6 full calibration
Named scan presets, visible criteria	Should	P4
Quantile dotplot (drill-down)	Should	P6
Sector small multiples	Should	P6
Paper portfolio + cost mirror (module 8)	Must (protective)	P6, gated on Academy M3
Curated top news · RS comparison chart · %K	Later	P6+ / backlog

STANDING REMINDERS WHILE BUILDING

The Research Report's Part 8 commitments function as the PR checklist: no directional forecasts, no bare percentages, no trending surfaces, mechanical voice, folklore labeled. When a tempting feature conflicts with the evidence chapters, the evidence chapters win — that is what they are for. TDD applies throughout (indicators, base rates, the verification gate, display rules, the sizing cap); the acceptance criteria above are deliberately phrased as tests.

Risks & mitigations

Ordered by expected pain. The catastrophic ones are operational silence and quiet dishonesty — both engineered against, not hoped against.

Risk	Severity	Mitigation
Silently dead pipeline — schedules can be auto-disabled (documented for public repos after 60 idle days; unverified for private), delayed, or occasionally dropped; cron failures page nobody by default; the app would quietly serve stale data	● HIGH	Zero-cost heartbeat step removes the auto-disable question; failure e-mails catch crashes; the dead-man check runs on the pipeline's own cron schedule (no weekend false alarms) with the holiday path pinging success — absence of success alerts by ~9:10pm EDT / 8:10pm EST, the same evening. Freshness timestamp rendered on the Desk; <code>workflow_dispatch</code> is the manual recovery.
GitHub Actions free-minutes exhaustion mid-month would kill the cron exactly like a dead pipeline	● MEDIUM	Designed out: the two-job split (~50 min/night ≈ 1,050/mo) leaves ~45% headroom of the 2,000 free minutes for reruns and backfills; a blocking single-job design (~1,650/mo) was explicitly rejected. If usage ever creeps, the Hetzner escape hatch is pre-priced.
Oracle drift — the product slowly grows prediction features under enthusiasm pressure	● MEDIUM	The non-negotiables box (Part 1) + Research Report Part 8 as the standing PR checklist; the track-record page makes any drift publicly visible in the app's own Brier score.
Indicator or base-rate math is simply wrong — hand-rolled code feeding every displayed number	● MEDIUM	Unit tests against published reference values; one-time cross-check against TA-Lib in CI; the one-bar-shift lookahead guard has its own failing-test proof; Wilson CI implementation tested against textbook cases.
Supabase free pauses after 7 idle days if the pipeline itself is broken — one bug becomes two outages	● MEDIUM	Dead-man alert surfaces a broken pipeline the same evening, not seven days later; if the coupling still rankles, Neon free has no pause concept and Prisma makes the swap a connection-string change.
Free-tier rug-pulls — precedent is real (Alpha Vantage 500→25 req/day; Finnhub removed free candles)	● MEDIUM	Repository-pattern adapters make any provider swap a one-module change; the upgrade ladder is pre-researched with verified prices so the response is a same-week migration.
LLM hallucination in the briefing — a fabricated number read before the open could influence a real decision	● MEDIUM	Three-layer defense: LLM never computes (numbers passed in, used verbatim); per-claim source-ID citations enforced by schema; deterministic tolerance-based gate blocks unsourced numbers/tickers/dates. Residual ~0.7–1.5% grounded-hallucination rate contained by inline citations in the UI.
Alpaca data quality — IEX-only quotes differ slightly from the consolidated tape, and 7 years of history may be short for some base rates	● MEDIUM	For EOD closes the divergence is minor; monthly sample cross-check against Stooq/yfinance; Stooq backfill can extend depth. If quality or depth becomes the binding pain, Tiingo Power (\$30, 30+ yrs, best cleanliness reputation) or EODHD (\$19.99) is the targeted fix.

Risk	Severity	Mitigation
Scope creep / solo-maintainer fatigue — the Academy alone could swallow months	● MEDIUM	Phase gates with testable acceptance criteria; the widget→phase traceability table keeps design and build reconciled; P0–P3 delivers the daily ritual early so motivation compounds.
Licensing violation by accident — sharing the dashboard breaches every provider’s personal-use terms	● LOW	Basic Auth always on; preview deployments protected; FRED attribution and TradingView logo rendered; no public screenshots with data.
Batch API latency blows the evening deadline	● LOW	Job B collects ~65 minutes after submission (typical completion is well under an hour); if incomplete, cancel and extract the remainder synchronously (~3 min, ~\$0.18 extra; already-processed items still bill). Worst-case publish ~00:50 UTC = 8:50pm EDT / 7:50pm EST — inside the 9:00pm ET promise year-round.
US DST edges — the close shifts between 20:00 and 21:00 UTC across the year, and the user’s own clock shifts with it	● LOW	Designed out: every trigger is UTC-fixed and sits after the close in both regimes, so no cron is ever edited; the only visible effect is that listed local times read an hour earlier on the winter clock (the schedule shows both); <code>exchange_calendars</code> handles US holidays and half-days; <code>lib/time.ts</code> owns EDT/EST labeling and is tested across both transitions.
Unverified numbers in this blueprint — Hetzner ~\$5, R2 10GB free, healthchecks.io free tier, FMP month-to-month rates, Highcharts commercial range, the 60-day rule on private repos	● LOW	All flagged inline where used; none is load-bearing for the \$0 baseline (each has a same-tier substitute). Confirm at signup before depending on any.

9.1 Decision changers

Three futures would legitimately reopen parts of this blueprint. **Intraday trading:** minute bars multiply storage ~400×, demand real-time infrastructure (SIP entitlements, a relay for client-side quotes) and a different risk posture — a different product; the Research Report’s evidence argues against wanting it. **Live brokerage integration:** fully available to a US resident, so the gate is earned rather than legal — order routing, credential security, and a changed risk posture arrive with it; legitimate only after the paper track record demonstrates readiness, with every friction mechanism carried over. **More users than you:** every data license, the auth model, and Vercel Hobby terms break simultaneously — that is a commercial product with data-redistribution contracts, not an upgrade.

Sources

Official pages used for verification, with dates checked. Domain-evidence sources live in the Research Report's Part 11; this list covers platforms, pricing, licensing, and tooling.

-
- [1] Alpaca — Market Data API plans, limits, redistribution policy (verified 2026-07-08).

docs.alpaca.markets/docs/about-market-data-api · alpaca.markets/data · alpaca.markets/support/redistribute-alpaca-api

- [2] Finnhub — pricing (incl. embedded price map), API docs (candles premium-gating), Terms of Service (verified 2026-07-08).

finnhub.io/pricing · finnhub.io/docs/api · finnhub.io/terms-of-service

- [3] EODHD — pricing, API limits & call weighting, personal-vs-commercial license (verified 2026-07-08).

eodhd.com/pricing · eodhd.com/financial-apis/api-limits · eodhd.com/financial-apis/commercial-vs-personal-license-use

- [4] Tiingo — pricing, IEX documentation, fundamentals add-on, license scope (verified 2026-07-09).

tiingo.com/pricing · tiingo.com/documentation/general/overview

- [5] Massive (formerly Polygon.io, rebranded Oct 2025) — plan catalog, Market Data Terms (verified 2026-07-09).

massive.com/pricing · massive.com/terms/market_data_terms.pdf

- [6] Financial Modeling Prep — pricing plans and FAQ; annual-vs-monthly billing caveat (verified 2026-07-08).

site.financialmodelingprep.com/pricing-plans · site.financialmodelingprep.com/faqs

- [7] SEC EDGAR — accessing EDGAR data (rate limit, User-Agent), APIs, dissemination timing, public-domain status (verified 2026-07-08).

sec.gov/search-filings/edgar-search-assistance/accessing-edgar-data · sec.gov/search-filings/edgar-application-programming-interfaces

- [8] FRED — API docs, v2 launch (Nov 2025), rate limit, Terms of Use and attribution requirement (verified 2026-07-08/09, partially via Internet Archive snapshots of official pages).

fred.stlouisfed.org/docs/api/fred/ · fred.stlouisfed.org/docs/api/terms_of_use.html

- [9] Marketaux — pricing, documentation, FAQ, ToS (verified 2026-07-08; snippets-only and per-tier request/article caps confirmed).

marketaux.com/pricing · marketaux.com/documentation

- [10] yfinance — repository and docs (unofficial status, personal-use caveat, rate-limit issue threads); Yahoo exchange-delay page SLN2310 (verified 2026-07-08).

github.com/ranaroussi/yfinance · help.yahoo.com/kb/SLN2310.html

- [11] TradingView — Lightweight Charts v5 repo, Apache-2.0 license + attribution; Advanced Charts free-license agreement (personal use excluded).
- github.com/tradingview/lightweight-charts · tradingview.com/advanced-charts/
-
- [12] Highcharts — EULA clarification for personal/non-commercial use; shop pricing.
- highcharts.com/blog/news/our-new-eula-makes-free-usage-clearer/
-
- [13] ApexCharts v5 — dual-license LICENSE file (no longer MIT; community license for individuals).
- github.com/apexcharts/apexcharts.js LICENSE
-
- [14] npm registry — peer-dependency verification (2026-07-08): `lightweight-charts@5.2.0`, `lightweight-charts-react-components@2.5.0`, `recharts@3.9.2`, `echarts@6.1.0`, `echarts-for-react@3.0.6`, `@visx/xychart@4.0.0`, `chartjs-chart-financial@0.2.1` (dormant).
-
- [15] Vercel — function max duration (Hobby 300s, Pro 800s/extended beta), cron jobs (Hobby once-daily, hour-imprecise; 100 crons/project), Python runtime status (verified 2026-07-08).
- vercel.com/docs/functions/configuring-functions/duration · vercel.com/docs/cron-jobs
-
- [16] GitHub — Actions billing (free minutes, private repos), 6-hour job limit; schedule auto-disable documented for public repositories (private-repo behavior unverified); community documentation of cron jitter and dropped events.
- docs.github.com/en/billing/concepts/product-billing/github-actions
-
- [17] Supabase — free-project pausing policy; connection modes (direct IPv6-only on free; Supavisor session/transaction poolers).
- supabase.com/docs/guides/platform/free-project-pausing
-
- [18] Neon — pricing and plans (100 CU-hours/month free; no pause-on-inactivity concept comparable to Supabase's).
- neon.com/pricing
-
- [19] Cloudflare — Workers platform limits (basis for elimination); R2 free-tier figure flagged unverified — confirm at signup.
- developers.cloudflare.com/workers/platform/limits
-
- [20] Anthropic — model pricing and Batch API (official pricing cache dated 2026-06-24: Haiku 4.5 \$1/\$5, Sonnet 5 \$3/\$15 with \$2/\$10 intro through 2026-08-31, Opus 4.8 \$5/\$25 per MTok; Batches –50%; structured outputs). Reconfirm at build time.
- platform.claude.com/docs/en/pricing
-
- [21] Python tooling status — pandas-ta archival deadline (2026-07-01) and pandas-ta-classic fork; TA-Lib activity (June 2026); vectorbt 1.1.0 (2026-07-05); exchange_calendars 4.13.2.
- pypi.org/project/pandas-ta · github.com/TA-Lib · pypi.org/project/exchange-calendars
-
- [22] DuckDB/Polars benchmarks and the Polars-prep + DuckDB-SQL pattern (codecentric large-Parquet memory benchmark; 2026 embedded-database roundups).
-
- [23] Parquet-vs-database storage comparisons for OHLCV research workloads (ts_db comparison; Concretum Group writeup).
-

[24] Financial-LLM grounding & verification research — FAITH-style multivariate-calculation collapse; FinGround/FinVerBench atomic-claim verification; 2025–26 grounded-hallucination leaderboards (0.7–1.5%); LLM citation-validation $\approx 38\%$ accuracy.

[25] Local-vs-API LLM TCO comparisons (2026): self-hosted $\sim \$126\text{--}233/\text{mo}$ all-in vs $\sim \$7\text{--}11/\text{mo}$ API bill at this volume.

[26] Hetzner CX22 pricing — third-party trackers (€4.49–4.59 after Apr 2026 adjustment) —

~~unverified~~ confirm at checkout. Backblaze B2 as R2 fallback.

[27] healthchecks.io — dead-man-switch cron monitoring with cron-schedule-matched checks; free tier taken from research,

~~unverified~~ any equivalent cron monitor works.

Blueprint compiled July 2026; revised after a six-lens adversarial review (quantitative, data-source, architecture, beginner-protection, UX, and completeness passes) whose accepted findings are incorporated throughout. Provider pricing verified against official pages on 8–9 July 2026 by dedicated verification passes (evidence URLs above); items that could not be confirmed on official pages are explicitly flagged *unverified* wherever they appear. Companion volume: *myStockMarket — Research Report* (domain evidence and product design).